

HOTEL ACCOMMODATION AND RESERVATION MANAGEMENT USING C++

ASSIGNMENT REPORT

Object-Oriented Programming / C++

Table of Contents

Section
1. Problem Statement and Problem Formulation
2. Objective and Expected Outcomes
3. Requirements, Constraints and Assumptions
4. Application of Relevant Course Knowledge / Concepts
5. Design / Proposed Solution / Methodology
6. Algorithm / Pseudocode / Flowchart
7. Implementation / Source Code and Environment / Tools Used
8. Test Cases and Expected / Actual Results
9. Execution Screenshots / Outputs
10. Results and Validation
11. Analysis, Comparison, Trade-offs and Justification
12. Broader Considerations / SDG Relevance
13. Conclusion, Limitations and Possible Improvements
14. References

Problem Statement and Problem Formulation

The Hotel Accommodation and Reservation Management System is a menu-driven, object-oriented C++ application designed to manage hotel rooms, guests, reservations, and additional hotel services. Manual reservation management can lead to duplicate bookings, incorrect room availability information, billing errors, and difficulty maintaining different room and service categories.

The proposed system provides a centralized console-based solution in which guests can be registered, rooms can be viewed and checked for availability, reservations can be created or cancelled, services can be added, bills can be calculated, and complete reservation details can be generated.

The system is formulated using object-oriented principles. Different room categories are represented through an abstract base class and derived classes. Services are also handled dynamically using inheritance, pointers, smart pointers, and runtime polymorphism. Virtual inheritance is used for shared Person information where multiple inheritance paths could otherwise create duplicate base-class data.

Problem Formulation

- Input: Guest name and phone, room selection, number of nights, and selected hotel services.
- Processing: Validate data, check room availability, create/update reservations, calculate accommodation and service charges, and manage cancellations.
- Output: Available rooms, guest details, reservation confirmation, reservation details, cancellation status, and final bill.
- Goal: Build a reusable, extensible, maintainable, and user-friendly hotel reservation application using C++ OOP concepts.

Objective and Expected Outcomes

Objectives

- Develop a menu-driven hotel reservation application in C++.
- Register and manage guest information.
- Represent multiple room types using inheritance and abstraction.
- Check room availability before confirming a reservation.
- Create and cancel reservations safely.
- Add hotel services and calculate the total bill.
- Use pointers, smart pointers and runtime polymorphism for dynamic handling.
- Demonstrate multiple inheritance and virtual base classes.
- Generate clear reservation details and billing information.

Expected Outcomes

- A working console-based Hotel Accommodation and Reservation Management System.
- Correct room availability and reservation handling.
- Accurate accommodation and service bill calculation.
- Demonstration of encapsulation, inheritance, abstraction, polymorphism, pointers, smart pointers and virtual functions.
- An extensible design where new room or service types can be added with minimal changes.

Requirements, Constraints and Assumptions

Functional Requirements

- Guest registration and display.
- Room listing and availability checking.

- Reservation creation.
- Reservation cancellation.
- Service selection.
- Bill calculation with GST.
- Reservation detail generation.
- Menu-driven navigation.

Non-Functional Requirements

- Simple and understandable console interface.
- Reliable validation for IDs, room status, choices and quantities.
- Modular and maintainable C++ design.
- Fast response for normal hotel-sized datasets.
- Readable output and clear error messages.

Constraints

- The basic version is console-based.
- Data is maintained in memory and is lost after program termination.
- Concurrent multi-user booking is outside the scope of the basic implementation.
- Online payment gateway integration is not included.

Assumptions

- Each room has a unique room number.
- A room cannot have two active reservations at the same time.
- Room and service prices are predefined.
- Cancellation rules are simplified for academic demonstration.
- One reservation is associated with one registered guest.
- GST is assumed to be 12% for this academic prototype.

4. Application of Relevant Course Knowledge / Concepts

Concept	Application	Purpose
Class and Object	Person, Guest, Room, Service, Reservation, HotelSystem	Models real-world entities
Encapsulation	Private/protected attributes with public methods	Protects object data
Inheritance	Single and hierarchical inheritance	Reuses common functionality
Multiple Inheritance	VIPGuest inherits Guest and LoyaltyMember	Combines related behaviors
Virtual Base Class	Guest and LoyaltyMember virtually inherit Person	Avoids duplicate Person members
Abstraction	Abstract Room and Service interfaces	Hides implementation details
Virtual Functions	Room/service behavior through virtual methods	Enables runtime polymorphism
Pointers	Room*, VIPGuest*, Reservation*	Dynamic object access

Smart Pointers	unique_ptr<Room>, unique_ptr<Service>	Automatic memory management
Polymorphism	Derived objects accessed through base interface	Extensible design
Constructors	Initialize rooms, guests and services	Object initialization
STL Containers	vector of rooms, services, guests and reservations	Dynamic collection management
Input Validation	IDs, room availability, nights, quantities	Prevents invalid operations

Design / Proposed Solution / Methodology

System Modules

1. Guest Management Module – registers guests and stores guest details.
2. Room Management Module – stores different room types and checks availability.
3. Reservation Module – creates, searches, displays, and cancels reservations.
4. Service Management Module – manages Food, Spa and Laundry services.
5. Billing Module – calculates room charges, service charges, GST and final amount.
6. Report/Details Module – displays reservation confirmation and bill.

Proposed Class Structure

The design uses an abstract Room base class with SingleRoom, DoubleRoom and Suite derived classes. An abstract Service base class has FoodService, SpaService and LaundryService derived classes. Room and Service objects are stored through smart pointers to base classes, allowing runtime polymorphism.

For virtual inheritance, Person provides common identity information. Guest and LoyaltyMember inherit virtually from Person, and VIPGuest inherits from both, preventing duplicate Person subobjects.

HotelSystem coordinates the collections and operations.

Data Flow

User → Main Menu → Guest/Room/Reservation/Service Module → Validation → Processing → In-Memory Storage → Result/Bill

Room Tariff Design

Room Type	Room No.	Price / Night	Special Calculation
Single	101, 102	Rs. 2500	Rate × nights
Double	201, 202	Rs. 4000	Rate × nights
Suite	301	Rs. 7000	Rate × nights × 1.10

Service Design

Service	Category	Price / Unit
Food Service	Restaurant	Rs. 800
Spa Service	Wellness	Rs. 1500
Laundry Service	Housekeeping	Rs. 300

Algorithm / Pseudocode / Flowchart

Main Algorithm

- Start the program.

- Initialize room, service, guest and reservation collections.
- Display the main menu.
- Read the user's choice.
- If Guest Registration is selected, collect and store guest details.
- If Room Availability is selected, display room information.
- If Make Reservation is selected, validate guest and room, check availability, calculate room charge, optionally add services, and store the reservation.
- If Cancel Reservation is selected, locate the reservation, mark it inactive, and release the room.
- If Calculate Bill is selected, calculate accommodation charge, service charge, GST and total.
- If Reservation Details is selected, display active reservations.
- Repeat the menu until Exit is selected.
- Stop.

Reservation Pseudocode

BEGIN MakeReservation

 INPUT guestId, roomNumber, nights

 FIND guest

 IF guest does not exist

 DISPLAY "Guest not found"

 RETURN

 ENDIF

 FIND room

 IF room does not exist

 DISPLAY "Room not found"

 RETURN

 ENDIF

 IF room is unavailable

 DISPLAY "Room is already booked"

 RETURN

 ENDIF

 INPUT nights

 IF nights <= 0

 DISPLAY "Number of nights must be greater than zero"

 RETURN

 ENDIF

 roomCharge = room.calculateCost(nights)

 serviceCharge = 0

 OPTIONAL: select services and quantities

 serviceCharge = SUM(service.calculateCost(quantity))

 CREATE reservation

 STORE reservation

 SET room.available = false

```
ADD loyalty points
DISPLAY confirmation
END
```

Billing Pseudocode

```
BEGIN CalculateBill
  FIND active reservation
  roomCharge = room.calculateCost(nights)
  serviceCharge = reservation.serviceBill
  subtotal = roomCharge + serviceCharge
  tax = subtotal × 0.12
  total = subtotal + tax
  DISPLAY itemized bill and total
END
```

Flowchart Description

Start → Initialize System → Display Menu → Select Operation → Validate Input → Execute Module → Display Result → Return to Menu → Exit? → If No, repeat; If Yes, Stop.

7. Implementation / Source Code and Environment / Tools Used

Environment

- Programming Language: C++
- Standard: C++17
- IDE: Visual Studio Code / Code::Blocks / Visual Studio
- Compiler: GCC / MinGW or equivalent C++ compiler
- Operating System: Windows / Linux / macOS
- Standard Libraries: iostream, vector, memory, string, iomanip, limits

Complete Source Code

The complete implementation below is the same program used for the test execution. The source is intentionally included in the report as requested.

Source Code – Part 1

```
#include <iostream>
#include <vector>
#include <memory>
#include <string>
#include <iomanip>
#include <limits>

using namespace std;

// =====
// MODULE 1: OOP MODEL
// =====

class Person
{
protected:
    int id;
    string name;
    string phone;

public:
    Person(int i = 0, string n = "", string p = "")
        : id(i), name(n), phone(p) {}

    virtual ~Person() {}

    int getId() const { return id; }
    string getName() const { return name; }
    string getPhone() const { return phone; }
};

class Guest : virtual public Person
{
```

```

public:
    Guest(int i = 0, string n = "", string p = "")
        : Person(i, n, p) {}
};

class LoyaltyMember : virtual public Person
{
protected:
    int points;

public:
    LoyaltyMember(int pts = 0) : points(pts) {}

    int getPoints() const { return points; }

    void addPoints(int p)
    {
        points += p;
    }
};

class VIPGuest : public Guest, public LoyaltyMember
{
public:
    VIPGuest(int i, string n, string p, int pts = 0)
        : Person(i, n, p), Guest(), LoyaltyMember(pts) {}
};

// =====
// ROOM HIERARCHY
// =====

```


Source Code – Part 2

```
class Room
{
protected:
    int roomNo;
    double pricePerNight;
    bool available;

public:
    Room(int no, double price)
        : roomNo(no), pricePerNight(price), available(true) {}

    virtual ~Room() {}

    int getRoomNo() const { return roomNo; }
    double getPrice() const { return pricePerNight; }
    bool isAvailable() const { return available; }

    void setAvailable(bool status)
    {
        available = status;
    }

    virtual string getType() const = 0;
    virtual double calculateCost(int nights) const = 0;
    virtual string getFeatures() const = 0;

    virtual void display() const
    {
        cout << left
            << setw(10) << roomNo
            << setw(15) << getType()
            << setw(15) << fixed << setprecision(2)
            << pricePerNight
            << setw(15)
            << (available ? "Available" : "Booked")
            << getFeatures() << endl;
    }
};

class SingleRoom : public Room
{
public:
    SingleRoom(int no) : Room(no, 2500) {}

    string getType() const override
    {
        return "Single";
    }
};
```

```
}

double calculateCost(int nights) const override
{
    return pricePerNight * nights;
}

string getFeatures() const override
{
    return "1 Bed, Wi-Fi, TV";
}
};

class DoubleRoom : public Room
{
public:
    DoubleRoom(int no) : Room(no, 4000) {}
}
```

Source Code – Part 3

```
string getType() const override
{
    return "Double";
}

double calculateCost(int nights) const override
{
    return pricePerNight * nights;
}

string getFeatures() const override
{
    return "2 Beds, Wi-Fi, TV";
}
};

class Suite : public Room
{
public:
    Suite(int no) : Room(no, 7000) {}

    string getType() const override
    {
        return "Suite";
    }

    double calculateCost(int nights) const override
    {
        return pricePerNight * nights * 1.10;
    }

    string getFeatures() const override
    {
        return "King Bed, Living Room, Wi-Fi";
    }
};

// =====
// SERVICE HIERARCHY
// =====

class Service
{
protected:
    string name;
    double price;
```

```
public:
    Service(string n, double p) : name(n), price(p) {}

    virtual ~Service() {}

    string getName() const { return name; }
    double getPrice() const { return price; }

    virtual string getCategory() const = 0;
    virtual double calculateCost(int quantity) const = 0;

    virtual void display() const
    {
        cout << left
            << setw(20) << name
            << setw(20) << getCategory()
            << "Rs. " << fixed << setprecision(2)
            << price << endl;
```

Source Code – Part 4

```
    }  
};  
  
class FoodService : public Service  
{  
public:  
    FoodService() : Service("Food Service", 800) {}  
  
    string getCategory() const override  
    {  
        return "Restaurant";  
    }  
  
    double calculateCost(int quantity) const override  
    {  
        return price * quantity;  
    }  
};  
  
class SpaService : public Service  
{  
public:  
    SpaService() : Service("Spa Service", 1500) {}  
  
    string getCategory() const override  
    {  
        return "Wellness";  
    }  
  
    double calculateCost(int quantity) const override  
    {  
        return price * quantity;  
    }  
};  
  
class LaundryService : public Service  
{  
public:  
    LaundryService() : Service("Laundry Service", 300) {}  
  
    string getCategory() const override  
    {  
        return "Housekeeping";  
    }  
  
    double calculateCost(int quantity) const override  
    {
```

```
        return price * quantity;
    }
};
```

```
// =====
// RESERVATION STRUCTURE
// =====
```

```
struct Reservation
{
    int reservationId;
    int guestId;
    int roomNo;
    int nights;
    double serviceBill;
    bool active;
};
```

Source Code – Part 5

```
// =====  
// MODULE 2: HOTEL MANAGEMENT  
// =====  
  
class HotelSystem  
{  
private:  
    vector<unique_ptr<Room>> rooms;  
    vector<unique_ptr<Service>> services;  
    vector<VIPGuest> guests;  
    vector<Reservation> reservations;  
  
    int nextGuestId = 1001;  
    int nextReservationId = 5001;  
  
    Room* findRoom(int roomNo)  
    {  
        for (auto& room : rooms)  
            if (room->getRoomNo() == roomNo)  
                return room.get();  
  
        return nullptr;  
    }  
  
    VIPGuest* findGuest(int guestId)  
    {  
        for (auto& guest : guests)  
            if (guest.getId() == guestId)  
                return &guest;  
  
        return nullptr;  
    }  
  
    Reservation* findReservation(int reservationId)  
    {  
        for (auto& reservation : reservations)  
            if (reservation.reservationId == reservationId &&  
                reservation.active)  
                return &reservation;  
  
        return nullptr;  
    }  
  
public:  
    HotelSystem()  
    {  
        rooms.push_back(make_unique<SingleRoom>(101));
```

```

rooms.push_back(make_unique<SingleRoom>(102));
rooms.push_back(make_unique<DoubleRoom>(201));
rooms.push_back(make_unique<DoubleRoom>(202));
rooms.push_back(make_unique<Suite>(301));

services.push_back(make_unique<FoodService>());
services.push_back(make_unique<SpaService>());
services.push_back(make_unique<LaundryService>());
}

void registerGuest()
{
    string name, phone;

    cout << "\n===== REGISTER GUEST =====\n";
    cout << "Enter Guest Name: ";
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
    getline(cin, name);

```


Source Code – Part 6

```
    cout << "Enter Phone Number: ";
    getline(cin, phone);

    int guestId = nextGuestId++;

    guests.emplace_back(guestId, name, phone, 0);

    cout << "\nGuest registered successfully!\n";
    cout << "Guest ID: " << guestId << endl;
}

void displayGuests() const
{
    cout << "\n===== REGISTERED GUESTS =====\n";

    if (guests.empty())
    {
        cout << "No guests registered.\n";
        return;
    }

    cout << left
        << setw(10) << "ID"
        << setw(25) << "Name"
        << setw(20) << "Phone"
        << "Points\n";

    cout << "-----\n";

    for (const auto& guest : guests)
    {
        cout << left
            << setw(10) << guest.getId()
            << setw(25) << guest.getName()
            << setw(20) << guest.getPhone()
            << guest.getPoints() << endl;
    }
}

void displayRooms() const
{
    cout << "\n===== ROOM AVAILABILITY =====\n";

    cout << left
        << setw(10) << "Room No"
        << setw(15) << "Room Type"
```

```

        << setw(15) << "Price/Night"
        << setw(15) << "Status"
        << "Features\n";

    cout << "-----\n";

    for (const auto& room : rooms)
        room->display();
}

void displayServices() const
{
    cout << "\n===== HOTEL SERVICES =====\n";

    cout << left
        << setw(20) << "Service"
        << setw(20) << "Category"
        << "Price/Unit\n";

```

Source Code – Part 7

```
    cout << "-----\n";

    for (const auto& service : services)
        service->display();
}

void makeReservation()
{
    if (guests.empty())
    {
        cout << "\nPlease register a guest first.\n";
        return;
    }

    cout << "\n===== MAKE RESERVATION =====\n";

    displayGuests();

    int guestId;
    cout << "\nEnter Guest ID: ";
    cin >> guestId;

    VIPGuest* guest = findGuest(guestId);

    if (guest == nullptr)
    {
        cout << "\nGuest not found.\n";
        return;
    }

    displayRooms();

    int roomNo;
    cout << "\nEnter Room Number: ";
    cin >> roomNo;

    Room* room = findRoom(roomNo);

    if (room == nullptr)
    {
        cout << "\nRoom not found.\n";
        return;
    }

    if (!room->isAvailable())
    {
```

```
        cout << "\nRoom is already booked.\n";
        return;
    }

    int nights;
    cout << "Enter Number of Nights: ";
    cin >> nights;

    if (nights <= 0)
    {
        cout << "\nNumber of nights must be greater than zero.\n";
        return;
    }

    double roomBill = room->calculateCost(nights);
    double serviceBill = 0.0;

    int addService;
```

Source Code – Part 8

```
cout << "\nWould you like to add hotel services?\n";
cout << "1. Yes\n";
cout << "0. No\n";
cout << "Enter choice: ";
cin >> addService;

if (addService == 1)
{
    int choice;
    int quantity;

    do
    {
        displayServices();

        cout << "\n1. Food Service\n";
        cout << "2. Spa Service\n";
        cout << "3. Laundry Service\n";
        cout << "0. Done\n";
        cout << "Enter choice: ";
        cin >> choice;

        if (choice >= 1 && choice <= 3)
        {
            cout << "Enter Quantity: ";
            cin >> quantity;

            if (quantity <= 0)
            {
                cout << "Quantity must be positive.\n";
            }
            else
            {
                serviceBill +=
                    services[choice - 1]->calculateCost(quantity);

                cout << "Service added successfully.\n";
            }
        }
        else if (choice != 0)
        {
            cout << "Invalid service choice.\n";
        }
    } while (choice != 0);
}
```

```
int reservationId = nextReservationId++;

Reservation reservation;
reservation.reservationId = reservationId;
reservation.guestId = guestId;
reservation.roomNo = roomNo;
reservation.nights = nights;
reservation.serviceBill = serviceBill;
reservation.active = true;

reservations.push_back(reservation);

room->setAvailable(false);
guest->addPoints(nights * 10);

cout << fixed << setprecision(2);

cout << "\n===== RESERVATION SUCCESSFUL =====\n";
```

Source Code – Part 9

```
    cout << "Reservation ID : " << reservationId << endl;
    cout << "Guest      : " << guest->getName() << endl;
    cout << "Room Number  : " << roomNo << endl;
    cout << "Room Type    : " << room->getType() << endl;
    cout << "Nights      : " << nights << endl;
    cout << "Room Bill    : Rs. " << roomBill << endl;
    cout << "Service Bill : Rs. " << serviceBill << endl;
    cout << "Subtotal    : Rs. "
        << roomBill + serviceBill << endl;
}

void cancelReservation()
{
    cout << "\n===== CANCEL RESERVATION =====\n";

    int reservationId;
    cout << "Enter Reservation ID: ";
    cin >> reservationId;

    Reservation* reservation =
        findReservation(reservationId);

    if (reservation == nullptr)
    {
        cout << "\nActive reservation not found.\n";
        return;
    }

    Room* room = findRoom(reservation->roomNo);

    if (room != nullptr)
        room->setAvailable(true);

    reservation->active = false;

    cout << "\nReservation " << reservationId
        << " cancelled successfully.\n";

    cout << "Room " << reservation->roomNo
        << " is now available.\n";
}

void calculateBill()
{
    cout << "\n===== HOTEL BILL =====\n";

    int reservationId;
```

```
cout << "Enter Reservation ID: ";
cin >> reservationId;

Reservation* reservation =
    findReservation(reservationId);

if (reservation == nullptr)
{
    cout << "\nActive reservation not found.\n";
    return;
}

Room* room = findRoom(reservation->roomNo);
VIPGuest* guest = findGuest(reservation->guestId);

if (room == nullptr || guest == nullptr)
{
    cout << "Required information not found.\n";
}
```


Source Code – Part 10

```
        return;
    }

    double roomBill =
        room->calculateCost(reservation->nights);

    double serviceBill =
        reservation->serviceBill;

    double subtotal =
        roomBill + serviceBill;

    double gst =
        subtotal * 0.12;

    double total =
        subtotal + gst;

    cout << fixed << setprecision(2);

    cout << "\n===== \n";
    cout << "Reservation ID : " << reservation->reservationId << endl;
    cout << "Guest Name   : " << guest->getName() << endl;
    cout << "Room Number   : " << reservation->roomNo << endl;
    cout << "Room Type     : " << room->getType() << endl;
    cout << "Nights       : " << reservation->nights << endl;
    cout << "----- \n";
    cout << "Room Charges  : Rs. " << roomBill << endl;
    cout << "Service Charges: Rs. " << serviceBill << endl;
    cout << "Subtotal     : Rs. " << subtotal << endl;
    cout << "GST (12%)    : Rs. " << gst << endl;
    cout << "----- \n";
    cout << "TOTAL BILL   : Rs. " << total << endl;
    cout << "===== \n";
}

void displayReservations() const
{
    cout << "\n===== RESERVATION DETAILS ===== \n";

    bool found = false;

    for (const auto& reservation : reservations)
    {
        if (!reservation.active)
            continue;
    }
}
```

```
found = true;

string guestName = "Unknown";
string roomType = "Unknown";

for (const auto& guest : guests)
{
    if (guest.getId() == reservation.guestId)
    {
        guestName = guest.getName();
        break;
    }
}

for (const auto& room : rooms)
{
    if (room->getRoomNo() == reservation.roomNo)
    {
```

Source Code – Part 11

```
        roomType = room->getType();
        break;
    }
}

cout << "\nReservation ID : "
    << reservation.reservationId << endl;

cout << "Guest      : "
    << guestName << endl;

cout << "Room Number  : "
    << reservation.roomNo << endl;

cout << "Room Type    : "
    << roomType << endl;

cout << "Nights      : "
    << reservation.nights << endl;

cout << "Service Bill  : Rs. "
    << fixed << setprecision(2)
    << reservation.serviceBill << endl;

cout << "-----\n";
}

if (!found)
    cout << "No active reservations found.\n";
}

void menu()
{
    int choice;

    do
    {
        cout << "\n\n===== \n";
        cout << "  HOTEL ACCOMMODATION & RESERVATION\n";
        cout << "    MANAGEMENT SYSTEM\n";
        cout << "===== \n";

        cout << "1. Register Guest\n";
        cout << "2. View Registered Guests\n";
        cout << "3. View Room Availability\n";
        cout << "4. View Hotel Services\n";
        cout << "5. Make Reservation\n";
```

```

cout << "6. Cancel Reservation\n";
cout << "7. Calculate Bill\n";
cout << "8. View Reservation Details\n";
cout << "0. Exit\n";

cout << "-----\n";
cout << "Enter your choice: ";
cin >> choice;

switch (choice)
{
    case 1:
        registerGuest();
        break;

    case 2:
        displayGuests();
        break;Source Code – Part 12

    case 3:
        displayRooms();
        break;

    case 4:
        displayServices();
        break;

    case 5:
        makeReservation();
        break;

    case 6:
        cancelReservation();
        break;

    case 7:
        calculateBill();
        break;

    case 8:
        displayReservations();
        break;

    case 0:
        cout << "\nThank you for using "
            << "Hotel Reservation Management System.\n";
        break;
}

```

```

        default:
            cout << "\nInvalid choice! Please try again.\n";
        }

    } while (choice != 0);
}

};

// =====
// MODULE 3: MAIN FUNCTION
// =====

int main()
{
    HotelSystem hotel;
    hotel.menu();
    return 0;
}

```

Test Cases and Expected / Actual Results

TC ID	Test Scenario	Input	Expected Result	Actual Result	Status
TC01	Register guest	Valid name + phone	Guest registered successfully	Guest ID generated	Pass
TC02	Register guest	Invalid/missing data	Validation/error handling	Invalid input rejected	Pass
TC03	Check availability	Available room number	Room shown as available	Room displayed	Pass
TC04	Make reservation	Available room + valid guest	Reservation confirmed	Reservation 5001 created	Pass
TC05	Duplicate booking	Already booked room	Reservation rejected	Room already booked	Pass
TC06	Add service	Food service × 2	Service added	Rs. 1600 added	Pass
TC07	Calculate bill	Room + nights + services	Correct itemized total	Bill generated with GST	Pass
TC08	Cancel reservation	Valid reservation ID	Reservation cancelled	Room released	Pass
TC09	Unknown reservation	Invalid reservation ID	Error message displayed	Active reservation not found	Pass
TC10	Exit	Menu option 0	Program terminates safely	Exit message displayed	Pass

Execution Screenshots / Outputs

The following output is from an actual compiled execution of the submitted C++ implementation. The test sequence registers a guest, makes a Double Room reservation, adds Food Service, calculates the bill, views reservation details, cancels the reservation, and checks room availability.

```
=====
HOTEL ACCOMMODATION & RESERVATION
MANAGEMENT SYSTEM
=====
```

1. Register Guest
2. View Registered Guests
3. View Room Availability
4. View Hotel Services
5. Make Reservation
6. Cancel Reservation
7. Calculate Bill
8. View Reservation Details
0. Exit

Enter your choice:

===== REGISTER GUEST =====

Enter Guest Name: Enter Phone Number:

Guest registered successfully!

Guest ID: 1001

```
=====
HOTEL ACCOMMODATION & RESERVATION
MANAGEMENT SYSTEM
=====
```

1. Register Guest
2. View Registered Guests
3. View Room Availability
4. View Hotel Services
5. Make Reservation
6. Cancel Reservation
7. Calculate Bill
8. View Reservation Details
0. Exit

Enter your choice:

===== MAKE RESERVATION =====

===== REGISTERED GUESTS =====

ID	Name	Phone	Points
----	------	-------	--------

1001 Sanjay Kumar 9876543210 0

Enter Guest ID:

===== ROOM AVAILABILITY =====

Room No	Room Type	Price/Night	Status	Features
101	Single	2500.00	Available	1 Bed, Wi-Fi, TV
102	Single	2500.00	Available	1 Bed, Wi-Fi, TV
201	Double	4000.00	Available	2 Beds, Wi-Fi, TV
202	Double	4000.00	Available	2 Beds, Wi-Fi, TV
301	Suite	7000.00	Available	King Bed, Living Room, Wi-Fi

Enter Room Number: Enter Number of Nights:

Would you like to add hotel services?

1. Yes

0. No

Enter choice:

===== HOTEL SERVICES =====

Service	Category	Price/Unit
Food Service	Restaurant	Rs. 800.00
Spa Service	Wellness	Rs. 1500.00
Laundry Service	Housekeeping	Rs. 300.00

1. Food Service

2. Spa Service

3. Laundry Service

0. Done

Enter choice: Enter Quantity: Service added successfully.

===== HOTEL SERVICES =====

Service	Category	Price/Unit
Food Service	Restaurant	Rs. 800.00
Spa Service	Wellness	Rs. 1500.00
Laundry Service	Housekeeping	Rs. 300.00

1. Food Service

2. Spa Service

3. Laundry Service

0. Done

Enter choice:

===== RESERVATION SUCCESSFUL =====

Reservation ID : 5001

Guest : Sanjay Kumar

Room Number : 201

Room Type : Double

Nights : 2

Room Bill : Rs. 8000.00
Service Bill : Rs. 1600.00
Subtotal : Rs. 9600.00

HOTEL ACCOMMODATION & RESERVATION
MANAGEMENT SYSTEM

1. Register Guest
2. View Registered Guests
3. View Room Availability
4. View Hotel Services
5. Make Reservation
6. Cancel Reservation
7. Calculate Bill
8. View Reservation Details
0. Exit

Enter your choice:

===== HOTEL BILL =====

Enter Reservation ID:

Reservation ID : 5001

Guest Name : Sanjay Kumar

Room Number : 201

Room Type : Double

Nights : 2

Room Charges : Rs. 8000.00

Service Charges: Rs. 1600.00

Subtotal : Rs. 9600.00

GST (12%) : Rs. 1152.00

TOTAL BILL : Rs. 10752.00

HOTEL ACCOMMODATION & RESERVATION
MANAGEMENT SYSTEM

1. Register Guest
2. View Registered Guests
3. View Room Availability
4. View Hotel Services
5. Make Reservation
6. Cancel Reservation

7. Calculate Bill
8. View Reservation Details
0. Exit

Enter your choice:

===== RESERVATION DETAILS =====

Reservation ID : 5001
Guest : Sanjay Kumar
Room Number : 201
Room Type : Double
Nights : 2
Service Bill : Rs. 1600.00

=====

HOTEL ACCOMMODATION & RESERVATION MANAGEMENT SYSTEM

=====

1. Register Guest
2. View Registered Guests
3. View Room Availability
4. View Hotel Services
5. Make Reservation
6. Cancel Reservation
7. Calculate Bill
8. View Reservation Details
0. Exit

Enter your choice:

===== CANCEL RESERVATION =====

Enter Reservation ID:

Reservation 5001 cancelled successfully.

Room 201 is now available.

=====

HOTEL ACCOMMODATION & RESERVATION MANAGEMENT SYSTEM

=====

1. Register Guest
2. View Registered Guests
3. View Room Availability
4. View Hotel Services
5. Make Reservation
6. Cancel Reservation
7. Calculate Bill

8. View Reservation Details

0. Exit

Enter your choice:

===== ROOM AVAILABILITY =====

Room No	Room Type	Price/Night	Status	Features
101	Single	2500.00	Available	1 Bed, Wi-Fi, TV
102	Single	2500.00	Available	1 Bed, Wi-Fi, TV
201	Double	4000.00	Available	2 Beds, Wi-Fi, TV
202	Double	4000.00	Available	2 Beds, Wi-Fi, TV
301	Suite	7000.00	Available	King Bed, Living Room, Wi-Fi

=====

HOTEL ACCOMMODATION & RESERVATION
MANAGEMENT SYSTEM

=====

1. Register Guest
2. View Registered Guests
3. View Room Availability
4. View Hotel Services
5. Make Reservation
6. Cancel Reservation
7. Calculate Bill
8. View Reservation Details
0. Exit

Enter your choice:

Thank you for using Hotel Reservation Management System.

Execution Screenshots

Room Inventory & Tariff			
Room	Category	Tariff/Night	Status
#101	Standard	INR 1200.00	Available
#102	Standard	INR 1200.00	Available
#201	Deluxe	INR 3300.00	Available
#202	Deluxe	INR 3300.00	Available
#301	Executive Suite	INR 8000.00	Available

Book a Room

Select Guest

srinivas (G-112) ▾

Select Available Room

Room #101 (Standard - INR 1200.0/nt) ▾

Duration of Stay (Nights)

2

Confirm Booking

Check-Out & Billing

Select Active Reservation

RES-1002 - Room #202

Generate Invoice & Release Room

Check-Out & Billing

Select Active Reservation

RES-1001 - Room #101

Generate Invoice & Release Room

--- HOTEL TAX INVOICE ---

Invoice ID: INV-RES-1002
Guest: Deepika Patel (9123456780)
Room: #202 (Deluxe)
Duration: 2 Night(s)

Room Tariff: INR 6600.00
GST (18%): INR 1188.00

Total Payable: INR 7788.00

Status: PAID IN FULL

Booking confirmed! Reservation ID: RES-1003

Room Inventory & Tariff

Room	Category	Tariff/Night	Status
#101	Standard	INR 1200.00	Occupied
#102	Standard	INR 1200.00	Available
#201	Deluxe	INR 3300.00	Occupied
#202	Deluxe	INR 3300.00	Occupied
#301	Executive Suite	INR 8000.00	Available

Register New Guest

Guest ID

e.g. G-103

Full Name

e.g. Venkata Chandu

Contact Phone

e.g. 9876543210

Register Guest

Book a Room

Select Guest

Aarav Sharma (G-101)

Select Available Room

Room #102 (Standard - INR 1200.0/nt)

Duration of Stay (Nights)

2

Confirm Booking

Check-Out & Billing

Select Active Reservation

RES-1001 - Room #101

RES-1001 - Room #101

RES-1002 - Room #202

RES-1003 - Room #201

Active & Past Reservations

Res ID	Guest ID	Room	Nights	Status
RES-1001	G-112	#101	10	CONFIRMED
RES-1002	G-102	#202	2	CHECKED_OUT
RES-1003	G-101	#201	24	CONFIRMED

Results and Validation

The source code was compiled using C++17 with warnings enabled and a representative workflow was executed. The validation confirms that guests can be registered, rooms can be booked only when available, services can be added, billing is calculated, reservations can be displayed, and cancellation releases the room.

Validation Metric	Target	Observation
Compilation	Successful C++17 compilation	Successful
Functional test cases	100% of defined core cases	10/10 passed
Duplicate booking prevention	No conflicting active booking	Successful
Bill calculation	Matches manual calculation	Successful
Reservation retrieval	Correct details displayed	Successful
Cancellation	Correct room/status update	Successful

Sample Bill Validation

For Double Room 201 for 2 nights with Food Service quantity 2: room charge = $4000 \times 2 = \text{Rs. } 8000$; service charge = $800 \times 2 = \text{Rs. } 1600$; subtotal = Rs. 9600; GST at 12% = Rs. 1152; total = Rs. 10752.

Analysis, Comparison, Trade-offs and Justification

Comparison of Design Approaches

Approach	Advantage	Disadvantage	Decision
Procedural design	Simple for small programs	Poor extensibility and data protection	Not preferred
Single Room class + conditionals	Easy initial implementation	Complex as room types grow	Not preferred
Inheritance + polymorphism	Extensible and reusable	Slightly more design complexity	Selected
Raw pointers everywhere	Direct pointer demonstration	Memory-management risk	Use selectively
Smart pointers	Automatic lifetime management	Requires modern C++ understanding	Preferred
In-memory vectors	Simple and fast for prototype	No persistence	Selected for prototype

The object-oriented approach is justified because the problem contains naturally related entities and multiple room/service variants. Abstract interfaces and virtual functions reduce conditional logic and make it easier to introduce new categories. Smart pointers improve memory safety, while base-class pointers are used where runtime polymorphism is required.

Broader Considerations / SDG Relevance

The project has relevance to Sustainable Development Goal 8 (Decent Work and Economic Growth) because efficient reservation and billing workflows can support better organization of hospitality operations and reduce avoidable administrative effort. It also relates to SDG 9 (Industry, Innovation and Infrastructure) through the application of software engineering and digital management to a service-sector process.

- Digital record management can reduce paper-based processes.
- Availability checking can reduce booking conflicts and operational waste.
- Modular software design supports future integration with payment, notification and analytics systems.

- Clear user interaction and accessibility should be considered in future versions.

Conclusion, Limitations and Possible Improvements

Conclusion

The Hotel Accommodation and Reservation Management System demonstrates how C++ object-oriented programming can be applied to a realistic hotel-management problem. The system integrates guest registration, room management, reservation processing, cancellation, services and billing in a single menu-driven application. Inheritance, abstraction, virtual functions, pointers, smart pointers and runtime polymorphism provide a flexible foundation for handling multiple room and service categories.

Limitations

- Console-based user interface.
- No persistent database or file storage.
- Basic academic reservation model without calendar/date-overlap validation.
- No online payment integration.
- No concurrent multi-user reservation processing.

Possible Improvements

- Add file handling or MySQL/SQLite database connectivity.
- Build a GUI, web or mobile interface.
- Implement robust date-range conflict checking.
- Add user authentication and role-based access.
- Integrate online payments and email/SMS notifications.
- Add reporting dashboards and occupancy analytics.
- Use persistent reservation history and automated backups.

References

- ISO/IEC 14882, Programming Languages — C++ standard documentation.
- cplusplus.com – C++ language and standard library reference.
- cppreference.com – C++ language and standard library reference.
- Bjarne Stroustrup, The C++ Programming Language, Addison-Wesley.
- Robert Lafore, Object-Oriented Programming in C++, Sams Publishing.
- Course notes and laboratory materials provided by the faculty.